



我發過的那些

Composer & NPM 套件們：

開源入門與心得分享

Lucas Yang

- 喜好：宅在家刻網站 | 寫程式 | 看動漫 | 玩遊戲
- 工作：蜂巢數據科技的全端工程師
- 框架：Laravel / Vue.js / Tailwind CSS
- 套件：npm 30+ , Composer 10+
- 2020 年：PHP 也有 Day 56
- 2021 年：{Laravel x Vue}Conf Taiwan

 [ycs77](#)

 [star-note-lucas.me](#)



Agenda

- 從介紹開源和套件開始
- 分享我開發套件的動機和過程

你有沒有安裝/使用過套件？

```
npm install vue
```

```
composer require laravel/boost
```

從套件而了解到的「開源」

- 純 JS 套件
- jQuery
- Laravel
- Vue

狹義的開源 vs 廣義的開源

狹義的開源

- 單純將原始碼公開放置在 **GitHub** 或其他平台
- 不一定有 **License**
- 不一定接受外部貢獻

廣義的開源

- 原始碼公開，並附上明確的**授權條款**（MIT、Apache...）
- 允許他人**自由使用、修改、散布**
- **開放社群參與**（PR、Issues）
- 提供完整的**使用文件**（README、Docs）

什麼是套件 (Package / Plugin) ?

- **不修改本體程式碼**，即可擴充額外功能
- 可透過**套件管理工具**（npm / Composer）直接安裝，或可以手動下載安裝
- 有**版本管理**，方便升級、回滾與維護
- 套件 ≠ 模組（Module）——但今天不深究這個問題
 - **Node.js**：ESM 與 CommonJS 模組
 - **PHP**：PSR-4 與 Composer 套件
- Package ≠ Plugin——但今天也是不深究這個問題

Composer 套件和 NPM 套件的構成

Composer 套件

最少必要的檔案結構：

- ``composer.json`` — 套件的設定與描述檔
- ``src/`` — 主要程式碼目錄
- ``README.md`` — 使用說明文件

額外的檔案：

- ``LICENSE`` — 開源授權聲明
- `` .gitignore`` / `` .gitattributes`` — Git 版控設定
- ``tests/`` — 測試程式碼目錄

composer.json

```
{
    "name": "vendor/package-name",
    "description": "套件的描述",
    "type": "library",
    "license": "MIT",
    "require": {
        "php": "^8.2"
    },
    "require-dev": {
        "phpunit/phpunit": "^11.0"
    },
    "autoload": {
        "psr-4": {
            "Vendor\\PackageName\\": "src/"
        }
    },
    "autoload-dev": {
        "psr-4": {
            "Vendor\\PackageName\\Tests\\": "tests/"
        }
    },
    "minimum-stability": "stable",
    "prefer-stable": true
}
```

Composer 套件的 `src/` 目錄

src/MyClass.php

```
<?php

namespace Vendor\PackageName;

class MyClass
{
    public function doSomething(string $name): string
    {
        return "Hello, $name!";
    }
}
```

使用方式：

```
use Vendor\PackageName\MyClass;

$myClass = new MyClass();

echo $myClass->doSomething('Lucas'); // 輸出: Hello, Lucas!
```

Laravel 套件的 composer.json

```
{
    // ...
    "require": {
        "php": "^8.2",
        "illuminate/config": "^11.0 || ^12.0 || ^13.0",
        "illuminate/contracts": "^11.0 || ^12.0 || ^13.0"
    },
    "extra": {
        "laravel": {
            "providers": [
                "Vendor\\PackageName\\PackageNameServiceProvider"
            ],
            "aliases": {
                "PackageName": "Vendor\\PackageName\\Facades\\PackageName"
            }
        }
    },
    // ...
}
```

Laravel 套件的 `src/` 目錄

src/PackageNameServiceProvider.php

```
<?php

namespace Vendor\PackageName;

class PackageNameServiceProvider extends ServiceProvider
{
    public function register(): void
    {
        // 註冊服務
        // * 載入 Config
        // * 綁定服務容器 (Service Container)
    }

    public function boot(): void
    {
        // 啟動服務
        // * 載入路由
        // * 載入視圖
        // * 載入資源檔案 (CSS / JS)
    }
}
```

Laravel 套件的 `src/` 目錄

src/Facades/PackageName.php

```
<?php

namespace Vendor\PackageName;

use Illuminate\Support\Facades\Facade;

/**
 * @method static string doSomething(string $name)
 */
class PackageName extends Facade
{
    protected static function getFacadeAccessor()
    {
        return 'my-class'; // 對應 Service Container 綁定的
    }
}
```

使用方式：

```
use Vendor\PackageName\Facades\PackageName;

echo PackageName::doSomething('Lucas'); // 輸出：Hello, Lucas
```

發布 Composer 套件

- 註冊 **Packagist** 帳號，並連結 **GitHub Repo**
- 在 GitHub 上**建立 Release** 和 **Tag 新版本**
- **Packagist** 會自動抓取新版本的 commit hash，或手動觸發更新
- 使用 ``composer require vendor/package-name`` 安裝套件，會從 GitHub 下載對應版本的 zip 檔案

NPM 套件

最少必要的檔案結構：

- `package.json` —— 套件設定與描述檔
- `index.js` / `index.ts` —— 主要進入點 (entrypoint)
- `README.md` —— 使用說明文件

額外的檔案：

- `LICENSE` —— 開源授權聲明
- `.npmignore` —— 控制發佈到 npm 的內容

package.json

```
{  
  "name": "package-name",  
  "version": "1.0.0",  
  "description": "套件的描述",  
  "author": "Your Name",  
  "license": "MIT",  
  "main": "dist/index.js",  
  "files": ["dist"]  
}
```

NPM 套件 (ESM) 的 ``src/`` 目錄

src/index.js

```
export function doSomething(name) {  
  return `Hello, ${name}!`  
}
```

使用方式：

```
import { doSomething } from 'package-name'  
  
console.log(doSomething('Lucas')) // 輸出: Hello, Lucas!
```

src/index.js

```
export class MyClass {  
  doSomething(name) {  
    return `Hello, ${name}!`  
  }  
}
```

使用方式：

```
import { MyClass } from 'package-name'  
  
const myClass = new MyClass()  
  
console.log(myClass.doSomething('Lucas')) // 輸出: Hello, L
```

jQuery 套件

src/index.js

```
$.fn.myButton = function (options) {  
  const $button = $(this)  
    .text(options.content)  
    .css('color', 'red');  
  
  $button.on('click', function () {  
    alert(`You clicked: ${options.content}`);  
  });  
};
```

使用方式：

```
<button id="my-button-element"></button>  
  
<script>  
  $(function () {  
    $('#my-button-element').myButton({  
      content: 'Click Me',  
    });  
  });  
</script>
```

編譯 NPM 套件

- 使用打包工具將 **原始碼** 編譯成 **瀏覽器或 Node.js 可用的格式**
 - **TypeScript** 編譯成 **JavaScript**
 - **Vue SFC** 編譯成 **JavaScript 和 CSS**
 - 輸出 **ESM** 和 **CommonJS** 格式的 ``.js`` 檔案，以便在不同環境中使用
- 常見工具
 - **Webpack/Rollup**: 過去的主流打包工具
 - **esbuild**: 使用 Golang 編寫的超快速打包工具
 - **Vite**: 前端應用開發/打包工具，以使用 Rust 編寫的 Rolldown 作為核心
 - **tsdown**: 前端套件打包工具，以使用 Rust 編寫的 Rolldown 作為核心

發布 NPM 套件

- 註冊 **NPM 帳號**
- 在 GitHub 上**建立 Release** 和 **Tag 新版本**
- 使用 ``npm publish`` 發佈到 **NPM Registry**
- 或是透過 **Trusted publishing (OIDC)** 自動化發佈流程
- 使用 ``npm install package-name`` 安裝套件

接下來是，我和 **開源套件們** 的故事…

當遇見了程式

- 國中時期，第一次接觸到程式
- 家裡的一本 **ASP** 的書，和一本 **JavaScript** 的書
- 幫認識的人做了幾個小網站練習

音樂播放器



- 前端：jQuery
- 後端：ASP
- 資料庫：Microsoft Access
- 文件儲存：Google Drive
- 線上環境：ASP 的共享主機

jQuery 要怎麼寫? 用 jQuery Plugin 的寫法

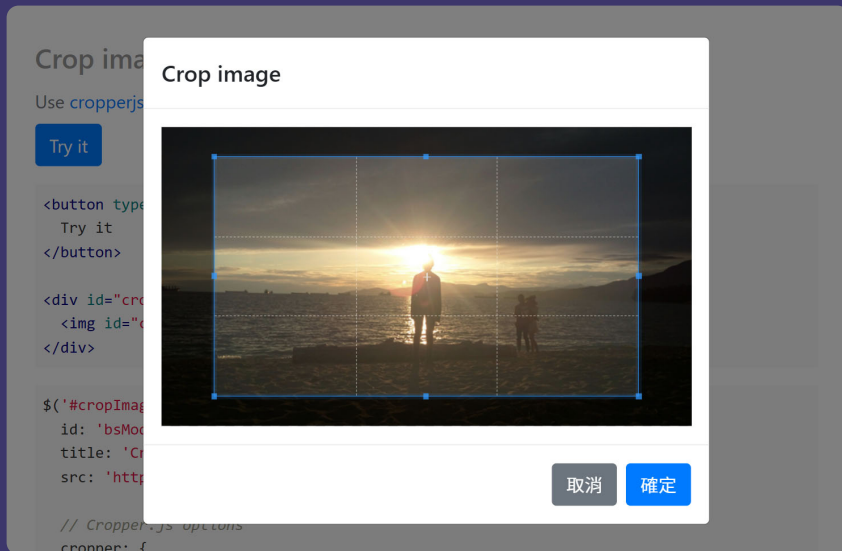
- 一開始只會寫 JavaScript
- 有一次幫忙改一個網站，要增加一個圖片輪播的功能
- 然後就用純 JavaScript 寫了一個簡單的圖片輪播
- 結果後來才在網路上看到有 jQuery 的圖片輪播 Plugin
- 然後就從這個 Plugin 的原始碼來學如何寫 jQuery Plugin...
- 以至於短時間內以為 jQuery Plugin 是寫 jQuery 的唯一方式...

在音樂播放器內使用 jQuery Plugin 寫法

```
<div id="player" class="player"></div>

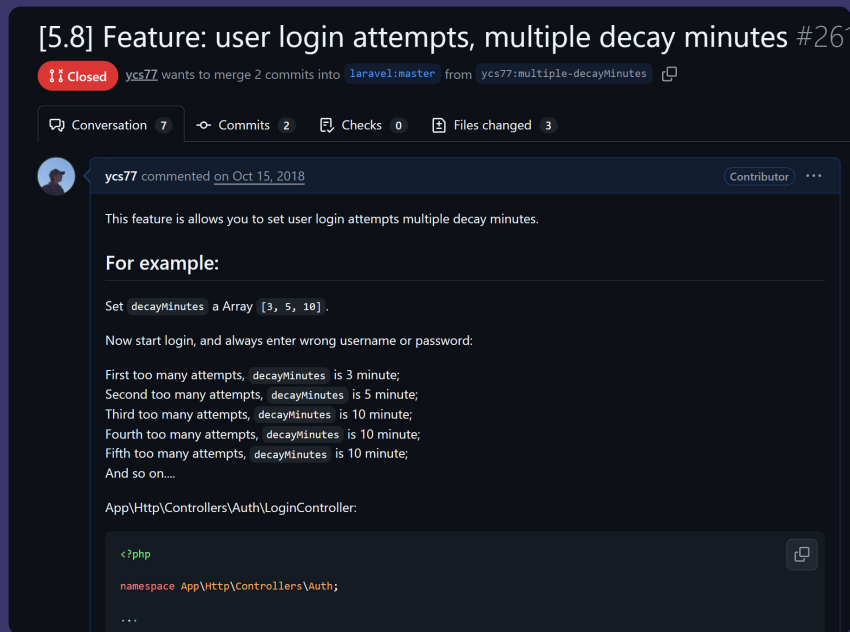
<script>
$(function(){
  $('#player').ycsaudio({
    id:    '<%=id%>',
    url:   '<%=url%>',
    title: '<%=title%><%if (listName ≠ 'null') {%>',
    list: [
      <%for (var i = 0; i < listAudioAryID.length - 1; i++) {%>{
        id:    '<%=listAudioAryID[i]%>',
        title: '<%=listAudioAryTitle[i]%>'
      },<%}%>
    ],
    listData: {id: '<%=listid%>', name: '<%=listName%>'}<%}if (autoplay = 'false' || autoplay = '0') {%>,
    autoplay: false<%}if (mes = 'me') {%>,
    me: 'me'<%}%>
  });
})
</script>
```

jQuery bsModel



- 第一個 npm 套件
- 可以快速建立 Bootstrap Modal
- 支援彈窗建立、圖片裁剪等功能
- 用於快速建立剪裁使用者 Avatar 的彈窗

給 Laravel 發 PR



- 線上 3D 模型購買平台的案子
- 登入機制需要增加類似 **指數退避** 的功能來阻止暴力破解
- Laravel 預設都是 **固定時間** 的鎖定機制，登入錯誤後都固定鎖定 **1 分鐘**
- 但是客戶希望鎖定時間可以失敗後會鎖 **3 分鐘**，下次 **5 分鐘**，下次 **10 分鐘**
- 但是 Laravel 的 ``src/Illuminate/Cache/RateLimiter.php`` 沒有這個功能
- 因此就自己實作了這個功能，然後發了一個 PR 給 Laravel

給 Laravel 發 PR



ludo237 commented on Oct 15, 2018

Contributor



Not tests? 🤔



TBlindaruk changed the title Feature: user login attempts, multiple decay minutes [5.8] Feature: user login attempts, multiple decay minutes on Oct 15, 2018



m1guelpf commented on Oct 15, 2018

Contributor



@yangchenshin77 Please add automated tests



給 Laravel 發 PR

[10.x] Fix "Text file busy" error when call deleteDirectory #48422

 Merged [taylorotwell](#) merged 1 commit into `laravel:10.x` from `ycs77:fix-text-file-busy`  on Sep 16, 2023

 Conversation **0**  Commits **1**  Checks **0**  Files changed **3**



[ycs77](#) commented on Sep 16, 2023

Contributor 

This PR is based on [#41433](#) and adds tests

If you call the `File::deleteDirectory($dir)` to delete a directory, you can see the files in the directory will be deleted, but the directory will keep it. If change the `@rmdir()` to `rmdir()` into `deleteDirectory()` it will be throws the `ErrorException: rmdir(/home/vagrant/packages/laravel-framework/vendor/orchestra/testbench-core/laravel/storage/app/public/testdir): Text file busy` error.

So this PR fixed the "Text file busy" error on call `File::deleteDirectory($dir)`, unset the `$items` in `deleteDirectory()` to release the directory lock before calling `rmdir()`, does not break existing features.

Reference PR: [thephpleague/flysystem#1103](#), [thephpleague/flysystem@d03f7e1](#)

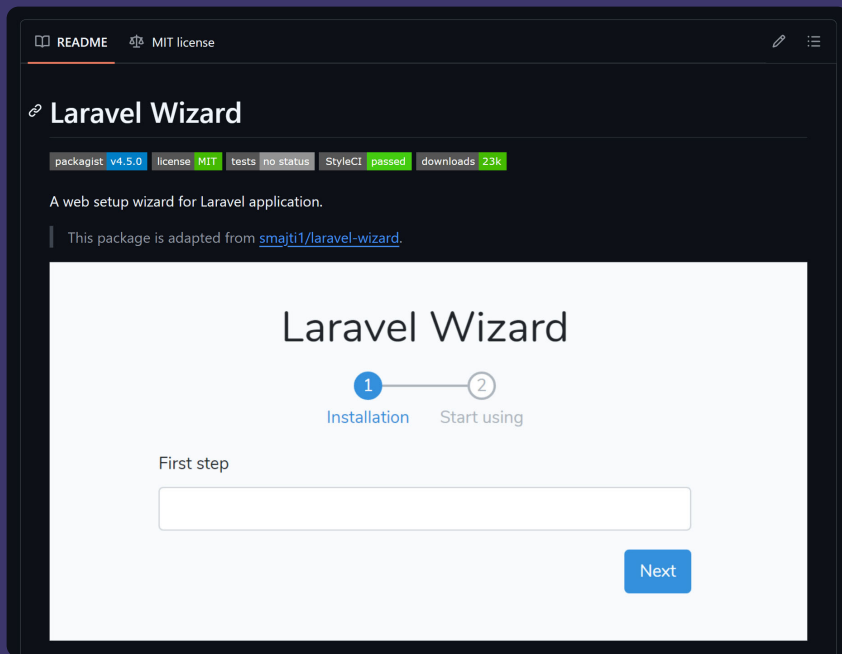


Laravel Newwebpay



- 藍新金流的**非官方 Laravel 套件**
- Fork 別人寫的版本來繼續維護，可以不用每個專案都要照著藍新的文件來寫一次金流串接程式了
- 改寫成了比較優雅的 Laravel 套件的寫法
- 但維護到後來時，也會開始覺得想要放棄…
- 最近正在重構到 v2 版本中…

Laravel Wizard *php*



- Laravel 安裝精靈套件
- 可以使用在註冊流程、購物流程等需要多步驟的流程中
- 曾經有個人在這個套件 Issues 問超多問題，甚至還問說能不能幫忙看他的程式碼

Headless UI Float



- 為 **Headless UI** 寫的**浮動定位套件**，為 Dropdown、Popover 等提供浮動定位的功能
- 支援 **React**、**Vue**、**Nuxt**
- GitHub **350+** Stars、npm 週下載量 **30,000+**
- 使用方式相對簡單，但內部實作相對複雜
- 需要使用 hack 的方式，因此如果上游的版本有改動，可能就會壞掉
- **現在已經 Archive 了**

Termwind 修復 CJK (中日韓) 文字排版 PR

- 從 2024/06 一直到 2025/10
- 瘋狂 at Francisco、nuno maduro、Taylor 三位和寄了好幾封 Email 才終於 Merge 了
- 雖然是 Termwind 的問題，但是會影響到 Laravel 的 Artisan CLI 和 Pest 的 CJK (中日韓) 文字排版
- 當然英文一直都是正常的
- 功能雖然能用，但是看著會不大舒服
- 原因是 ``mb_strlen()`` 會把中文當成 1 個字元來計算，需要改成使用 ``mb_strwidth()`` 才能正確計算中文的寬度 (學到冷知識了…)
- 順便認識了一位網友 James

Laravel CLI 修復

BEFORE

```
$ php artisan app:hello-world  
HELLO WORLD!  
Termwind ..... Hello World  
  
$ php artisan app:hello-world  
你好 世界。  
Termwind ..... 你好 世界  
  
$ php artisan app:hello-world  
ハロー ワールド。  
Termwind ..... ハロー ワ  
ールド
```

AFTER

```
$ php artisan app:hello-world  
HELLO WORLD!  
Termwind ..... Hello World  
  
$ php artisan app:hello-world  
你好 世界。  
Termwind ..... 你好 世界  
  
$ php artisan app:hello-world  
ハロー ワールド。  
Termwind ..... ハロー ワールド
```

Pest 測試畫面修復

BEFORE

```
$ vendor/bin/pest

PASS Tests\Unit\ChineseTest
✓ 你好 世界 1.00s
3s
✓ 你好 世界 你好 世界
1.01s

PASS Tests\Unit\EnglishTest
✓ Hello World! 1.03s
✓ Hello World! Hello World! 1.01s

PASS Tests\Unit\JapaneseTest
✓ ハロー ワールド
1.01s
✓ ハロー ワールド ハロー ワールド
1.01s

Tests: 6 passed (6 assertions)
Duration: 6.21s
```

AFTER

```
$ vendor/bin/pest

PASS Tests\Unit\ChineseTest
✓ 你好 世界 1.02s
✓ 你好 世界 你好 世界 1.01s

PASS Tests\Unit\EnglishTest
✓ Hello World! 1.03s
✓ Hello World! Hello World! 1.01s

PASS Tests\Unit\JapaneseTest
✓ ハロー ワールド 1.01s
✓ ハロー ワールド ハロー ワールド 1.01s

Tests: 6 passed (6 assertions)
Duration: 6.27s
```

AI 時代 還需要開發套件嗎？

apifable

- 今年年初在做一個案子時
- 串接 API 需要一直開 Swagger UI 來看 API 的資訊
- 那個 Swagger UI 的使用者體驗真的很差，載入速度巨慢
- 但同時又需要使用 AI Agent 來幫忙加速開發
- 因此開發此套件，讓 AI Agent 可以加速讀取 API 的資訊，來加速前端串接後端 API 的過程

apifable

- AI Agent 讀 OpenAPI Spec 的 MCP
- MCP 會去解析 ``openapi.yaml``
- Agent 就可以直接獲取 Endpoint、看 Schema、看 TypeScript 型別
- 加速前端串接後端 API 的過程
- 同時也實驗了 AI Agent 來幫忙開發套件的過程

列出可用 API

- apifable - get_spec_info (MCP)
└ {
 "title": "apifable - MCP Testing",
 "version": "1.0.0",
 ... +25 lines (ctrl+o to expand)
- 只有一個 tag，列出所有端點：
- apifable - list_endpoints_by_tag (MCP)(tag: "[1 - 身分驗證]")
└ {
 "tag": "[1 - 身分驗證]",
 "endpoints": [
 ... +33 lines (ctrl+o to expand)
- apifable - MCP Testing (v1.0.0)
 - Server: <http://localhost/api>
 - Auth: Bearer JWT

[1 - 身分驗證] (4 endpoints)

Method	Path	Summary
POST	/login	[1 - 001] 登入
POST	/logout	[1 - 002] 登出



給我 [1 - 003] 取得使用者資訊 的 API

- apifable - get_endpoint (MCP)(operationId: "auth.me")
└ {
 "method": "get",
 "path": "/me",
 ... +49 lines (ctrl+o to expand)

- GET /me - [1 - 003] 取得使用者資訊

認證: Bearer JWT (必要)

Request: 無參數、無 body

Response 200 - UserResource

```
{  
  "data": {  
    "id": 1, // integer, ID  
    "name": "John Doe", // string, 使用者名稱  
    "email": "test@example.com", // string, 使用者 Email  
    "created_at": "2026-01-01 00:00:00", // string | null, date-time  
    "updated_at": "2026-01-01 00:00:00" // string | null, date-time  
  }  
}
```

所有 data 內的欄位皆為必填。

Response 401 - Unauthenticated



為什麼會想要開發**套件**？

因為，開發**套件**的過程真的很好玩啊~